Esercitazione 07

17 aprile 2026

Lo scopo di questa esercitazione è quello di iniziare a prendere confidenza con la parallelizzazione di codice in C usando OpenMP. In particolare, riprendendo il progetto di lettura e scrittura di immagini in formato netpbm visto nell'esercitazione precedente, si chiede di generare delle trasformazioni a partire da un immagine di riferimento. L'idea è quella considerare ogni pixel come un punto di una matrice e di applicare una trasformazione a questo punto che dipende dai valori adiacenti.

Recap: struttura dei files netpbm

```c
struct _netpbm_image {
  int width;
  int height;
  int offset;
  int size;
  FILE * fd;
  char * data;
};
```

Dove:

- `width` e `height` rappresentano le dimensioni dell'immagine in termini di numero di pixel orizzontali e verticali.
- `offset` indica in quale punto del file (in byte) iniziano i dati dell'immagine (i.e., la prima posizione dopo l'intestazione del file).
- `size` indica la dimensione totale del file in byte.
- `fd` serve a tenere traccia del file da cui è stata letta l'immagine.
- `data` è un puntatore ai dati dell'immagine. In particolare negli indici da `0` a `offset-1` sarà contenuta l'intestazione e successivamente (da `offset` a `size-1`) i pixel dell'immagine memorizzati riga per riga. Ne segue che il pixel della riga y e colonna x sarà all'indice `y * width + x + offset`.

e i valori dei pixel contenuti in `data` sono rappresentati come numeri interi senza segno che nel nostro caso vanno da 0 a 255 (i.e., 8 bit per pixel).

Trasformazioni da implementare

Si vuole implentare delle trasformazioni tramite i così detti **filtri convoluzionali**. L'idea è quella di considerare ogni pixel come un punto di una matrice e di applicare una trasformazione a questo punto che dipende nell'intorno di un certo raggio.

Nell'esercitazione odierna, non verrà considerato il caso del padding (cioè gestire i pixel ai bordi dell'immagine), pertanto data un immagine di dimensioni $w \times h$ e un raggio r, il risultato della trasformazione sarà un'immagine di dimensioni

$(w - 2r) \times (h - 2r)$, dove primo pixel in posizione $(0, 0)$ è stato calcolando a partire dal pixel in posizione (r, r) dell'immagine di input.

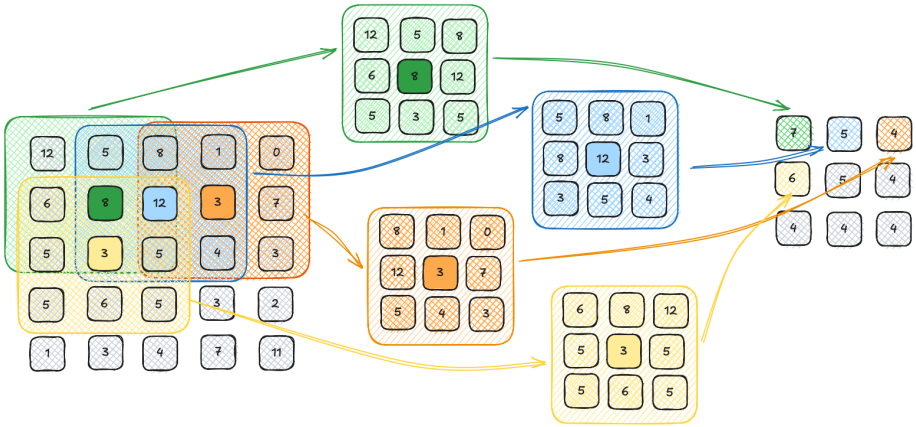

Figure 1: Esempio di filtro convoluzionale di raggio 1

Funzioni implementate

Viene già fornito i file `netpbm.h` e relativo `netpbm.c` con le seguenti funzioni implementate:

- `open_image`: apre un file in formato netpbm e popola una struttura `_netpbm_image` con le informazioni relative all'immagine.
- `close_image`: chiude un file in formato netpbm e libera le risorse associate.
- `pixel_at`: restituisce un puntatore al pixel in posizione (x, y) nell'immagine passata come argomento. Il puntatore è `NULL` se il pixel richiesto supera i limiti dell'immagine.
- `create_output_image`: crea un nuovo file in formato netpbm con le dimensioni corrette e lo apre restituendo una struttura `_netpbm_image` popolata con le informazioni relative all'immagine appena creata.

Viene altresì fornito il `main.c` che implementa le due funzioni a vostra disposizione:

- `benchmark_images(netpbm * image, int num_threads, int n_iter)`: esegue *n_iter* iterazioni delle trasformazioni delle immagini misurandone il tempo di esecuzione e calcolandone la media. Questo viene ripetuto scalando il numero di thread da 1 a *num_threads*.
- `test_functions(netpbm * image, int num_threads)`: chiama una sola volta le funzioni da implementare settando il numero di thread a *num_threads* (tipicamente il massimo supportato dalla macchina dove si esegue il codice).

Funzioni da implementare

Si chiede di implementare, e successivamente parallelizzare con `OpenMP`, le seguenti funzioni:

1. Blur filter

- `void blur_image(netpbm_ptr src, chart *path, netpbm_ptr dst`: applica un filtro di sfocatura all'immagine di input `src` e salva il risultato in `dst`. Dato un pixel in posizione (x, y), esendo r il raggio del filtro (controllato dalla variabile `RADIUS_BLUR` definita in `image_transform.h`), il valore nell'immagine `dst` sarà calcolato come la media (arrotondata per difetto) dei pixel nell'intorno:

$$dst(x, y) = \lfloor \frac{1}{(2r+1)^2} \sum_{i=-r}^{r} \sum_{j=-r}^{r} src(x+i, y+j) \rfloor$$

In questo caso si richiede un implementazione con un algoritmo "naive" che itera su ogni pixel e per ogni pixel itera su tutti i pixel nell'intorno:

```
// itera su tutti i pixel dell'immagine di input
for x = r to width - r:
  for y = r to height - r:
    // itera su tutti i pixel nell'intorno
    sum = 0
    for i = x-r to x+r:
      for j = y-r to y+r:
        sum += src(x+i, y+j)
    dst(x,y) = sum / (2r+1)^2
```

Nota: Si noti che in questo modo il numero di operazioni eseguite è dell'ordine di $\mathcal{O}(w \cdot h \cdot r^2)$, dove w e h sono rispettivamente la larghezza e l'altezza dell'immagine di input.

2. Edge highlight filter

- `void edge_highlight(netpbm_ptr src, chart *path, netpbm_ptr dst`: applica un filtro di evidenziazione dei bordi all'immagine di input `src` e salva il risultato in `dst`, immagine che in questo caso sarà in bianco e nero. Dato un pixel in posizione (x, y), esendo r il raggio del filtro (controllato dalla variabile `RADIUS_BORDER_HIGHLIGHT` definita in `image_transform.h`), il valore nell'immagine `dst` sarà calcolato come:

$$dst(x, y) = \begin{cases} 255 & \text{se } \text{avg}_r(x, y) \geq src(x+r, y+r) \\ 0 & \text{altrimenti} \end{cases}$$

In questo secondo esempio, simile al precedente tranne per il confronto finale tra la media dell'intorno, si richiede di evitare l'implementazione "naive" del quaruplo ciclo annidato.

Una implementazione molto comune consiste nell'introdurre una fase di preprocessing finalizzata a calcolare la tabella delle somme cumulative (prefix table). Data una matrice M di dimensioni $w \times h$, la tabella delle somme cumulative P è una matrice di dimensioni $(w+1) \times (h+1)$ dove la prima riga e la prima colonna sono inizializzate a zero e gli altri elementi sono calcolati come:

$$P(x, y) = \sum_{i=0}^{x} \sum_{j=0}^{y} M(i, j)$$

Per esempio:

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \Rightarrow P = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 3 & 6 \\ 0 & 5 & 12 & 21 \\ 0 & 12 & 27 & 45 \end{bmatrix}$$

Pagando un costo di $\mathcal{O}(w \cdot h)$ per costruire la tabella delle somme cumulative, è possibile calcolare la somma di qualsiasi sotto-matrice $r \times r$ in tempo costante richiedendo solo 4 accessi alla tabella P:

$$\sum_{i=-r}^{r} \sum_{j=-r}^{r} M(x+i, y+j) = D - C - B + A$$

dove:

- $A = P(x-r, y-r)$
- $B = P(x+r, y-r)$
- $C = P(x-r, y+r)$
- $D = P(x+r, y+r)$.

Si consiglia di implementare una funzione d'appoggio `build_prefix_sum(const unsigned char * data, int w, int h)` che costruisce la tabella delle somme cumulative a partire dai dati dell'immagine di input.

Nota: Siccome le immagini possono essere molto grandi, e sommano valori certamente positivi, il file d'interfaccia `image_transform.h` utilizza il tipo `unsigned long long` per rappresentare i valori della tabella delle somme cumulative, in modo da evitare problemi di overflow.

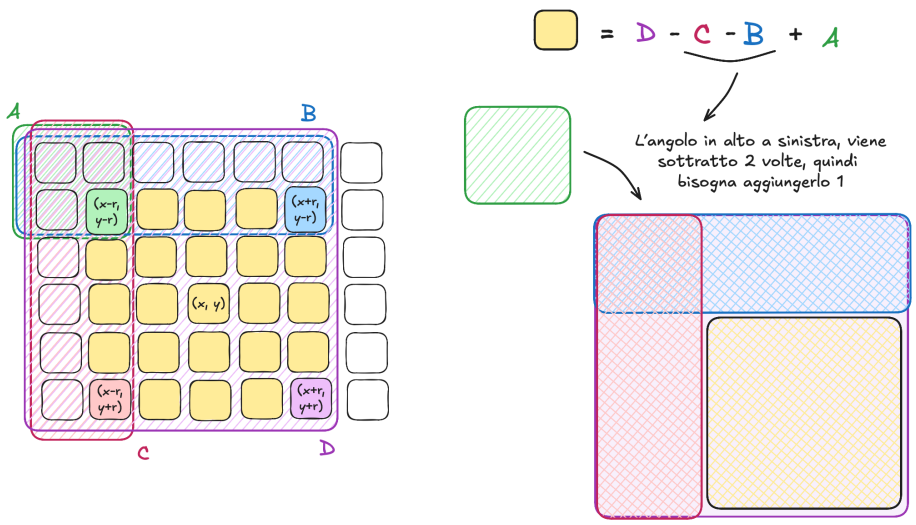

Figure 2: Visualizzazone grafica di come calcolare la somma di una sotto-matrice $r \times r$ usando la tabella delle somme cumulative